P3 GC Design Note: Lisp-2 Sliding Mark-Compact (Rust Runtime)

Neha Reddy Sarthak Dayal Jay Shim Rahik Sikder
CS 378H, Fall 2026

1 Choice and Heap Layout

We implemented the *Lisp-2 sliding mark-compact* collector in the Rust skeleton. We chose it because it uses the whole heap rather than half, and because its difficulty sits in one place, the pointer-fixup discipline, which we could enumerate and test directly. The reference Cheney collector stays in `rust/reference/` behind the `reference-cheney` Cargo feature, so we can diff against it.

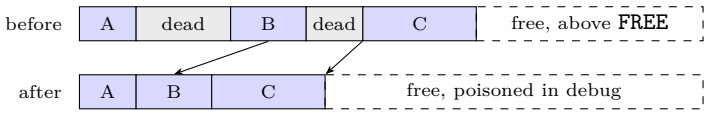
- **Heap.** One contiguous region with a bump pointer `FREE`, 16 MiB by default and overridden by `LO_HEAP_SIZE` at `lo_runtime_init`. Objects sit end to end, so the allocated prefix [base, FREE) can be walked linearly.
- **Allocation.** `lo_alloc` bumps `FREE`. If the object does not fit, it forces one collection and retries once, aborting through `lo_alloc_oom` (exit 137) only if the live set still leaves no room.
- **Sizes and offsets.** All sizes come from `heap_layout::object_size` (`align(instance_size)`, or `align(string_data_offset + length)` for a `String`), and every field offset comes from `offset_of` or the descriptors. Nothing is hard-coded to the 16-byte native or 12-byte WASM header.
- **Mark bit.** Bit 0 of `gc_bits`, clear between collections.
- **Side metadata.** Descriptors must stay intact throughout, since the heap walk reads them, so forwarding addresses are not stored in headers. `compaction::Scratch` instead holds a mark `worklist` and a table of `Relocation{old_offset, new_offset, size}`, both reserved *at init* and never during a collection.
- **No allocation during collection.** A collection never allocates, and re-entry is rejected by an `IN_COLLECTION` assertion.
- **Barrier.** `lo_gc_write_barrier` stays the skeleton's bare store. With one generation there is no old-to-young pointer to miss, so no remembered set or card table is needed.

2 Roots and Collection Passes

The only roots are the shadow-stack slots. The collector walks the `ShadowFrame` chain from `current_frame` and visits each frame's `num_roots` slots, including the static `lo_bindings` frame. Since P2's `gc_roots` spills every live `Ref` around each call, no register or native-stack scan is needed. `managed(p)` is true iff p is non-null and base $\le p <$ FREE, and every visited pointer passes through it. `LO_EMPTY_STRING`, literals, and descriptors live in static data, so they are never marked, moved, or rewritten, with no special case. The ABI supplies object-start pointers only, so an exact-start lookup suffices.

Phase	What it does
`mark`	Marks from the roots with an *explicit worklist* of heap offsets, so a 170,000-link chain cannot overflow the stack. The bit is set *before* pushing, so cycles and aliases enter once. Fields come from the descriptor's `pointer_offsets`, which need not be contiguous.
`compute_forwarding`	Walks the heap with `object_size`. Each marked object gets `new_offset` equal to the live bytes before it. Entries are appended in address order, so the table is sorted by construction.
`fix_references`	Rewrites every root slot and every pointer field of every live object, looking the pointee up by binary search. All fixups run while every object is still at its old address.
`slide`	Copies objects down in ascending order with the overlap-safe `ptr::copy`, clearing mark bits. Since new $\le$ old for every object, no unmoved object is overwritten.

Afterward `FREE` is set to the end of the live prefix. With `LO_GC_POISON=1` the freed tail is filled with `0xEF`, so a stale pointer fails loudly.



Fixup argument. Every managed pointer lives in one of these places. (1) Shadow-stack slots are rewritten. (2) Pointer

fields of live objects are rewritten, including objects that do not move, because their pointees may. (3) Non-heap targets are left alone by the range guard. (4) No interior runtime state exists, because string operations copy their inputs into a Rust `Vec` *before* allocating, `lo_cast_check` and `lo_instanceof` allocate nothing, and the worklist and relocation table hold offsets, not pointers. (5) `FREE` is reset. Dead objects are never visited, so their stale fields are never dereferenced.

3 Testing on Both Targets

`LO_HEAP_SIZE` shrinks the heap so programs collect early, `lo_gc_collect` forces collections, and `LO_GC_POISON=1` poisons freed space. **Rust tests** call the entry points directly. Unit tests in `gc.rs` run at 4 KiB, and `collector_probe` runs at 1 KiB with poisoning, on native and compiled to `wasm32` under `wasmrun`. The probe holds a two-node cycle from two frames across repeated collections, forces an overlapping slide of a 257-byte string, and allocates 1.1 million objects past the heap. `tests/run-collector.sh` runs all of it plus both release builds.

LO stress tests (suite header-contract format) are in `tests/gc-stress/`. There are five, and each allocates about 3 million objects, keeps live data under a quarter of its heap, and was confirmed to fail on a copy of our runtime with the named bug planted.

Test	Invariant (planted bug it catches)
`gc_01_unmoved_anchor_fixup`	Fixup rewrites fields of unmoved live objects (fixup skips objects whose address is unchanged)
`gc_02_overlapping_slide`	A slide shorter than the object is overlap-safe (copying high to low, or the tail block first)
`gc_03_survivors_then_die`	Survivors are reclaimed once dead (mark bits left set)
`gc_04_deep_list_marking`	Marking uses a worklist (recursive marking on a 170,000-link chain)
`gc_05_every_pointer_field`	Scan visits exactly the descriptor's offsets (first field only, last field skipped, contiguity assumed)

`run.sh` runs each program on WASM and native at the default heap, and on native at its declared heap. All five pass on both targets, and natively at their declared heaps. A WASM module has no environment, so `LO_HEAP_SIZE` cannot reach it and WASM always runs at 16 MiB. There, collections come from allocating past the heap and from direct `lo_gc_collect` calls.

4 Worst Bug

[TODO: Confirm this is the bug the team wants to name, since it is the only one the git history records.]
The worst bug was at the collector's edge and differed by target. A heap-exhaustion abort inside a *string* operation (`lo_string_concat`, `lo_string_new`, `lo_string_reverse`) printed the right message on both targets but exited 137 on native and **134** on WASM. `wasmrun` derives the exit code from the trap backtrace and maps a trap to 137 only if a `lo_alloc*` frame is on it, and our string operations aborted from their own out-of-memory paths. We found it by running tiny-heap tests on WASM. The fix routes *every* out-of-memory abort through one `#[cold] #[inline(never)] lo_alloc_oom`. One case remains, since OOM inside `lo_string_repeat` still exits 120 on WASM because `wasmrun` matches that name first.

String and type operations. The five `lo_string_*` operations are UTF-8 aware (codepoint-aware reversal, byte-wise comparison, exit 120 on a negative repeat count) and follow the root discipline above. `lo_cast_check` and `lo_instanceof` walk the descriptor `parent` chain, with null short-circuits and exit 101 naming both classes. `string_type_regression` tests both, including non-ASCII input.

Limitations. There is no heap-checker walk or GC log, only poisoning. Fixup costs a binary search per pointer rather than $O(1)$ forwarding, and the side tables reserve about twice the heap in address space.